



The Pipeline from Car Accidents to Public Transportation



Pedro Kawamura, Owen Seibold



Context

As the population in the United States grows, it is important to look for efficient modes of transportation. In doing so, it is even more crucial to consider the safety and productivity of our methods.

As a result, in this project we are looking at the car accident rates and how they might influence a transition of commuters from private to public transportation.

1. Downloaded Dataset (Subway/Bus lost time csv)

	A	B	C	D	E	F
1	Month	Department	Lost Time Accidents	Employees	Lost Time Accident Rate	
2	1/1/2021	Subways	104	28903	0.36	
3	1/1/2021	Buses	81	14827	0.55	
4	1/1/2021	Miscellaneous NYCT	8	4963	0.16	
5	1/1/2021	MTA Bus	17	3807	0.45	
6	2/1/2021	Subways	114	28822	0.4	
7	2/1/2021	Buses	75	14754	0.51	
8	2/1/2021	Miscellaneous NYCT	4	4943	0.08	
9	2/1/2021	MTA Bus	35	3790	0.92	
10	3/1/2021	Subways	104	28702	0.36	
11	3/1/2021	Buses	78	14690	0.53	
12	3/1/2021	Miscellaneous NYCT	0	4889	0	
13	3/1/2021	MTA Bus	25	3783	0.66	
14	4/1/2021	Subways	93	28702	0.32	
15	4/1/2021	Buses	89	14690	0.61	
16	4/1/2021	Miscellaneous NYCT	7	4889	0.14	
17	4/1/2021	MTA Bus	24	3752	0.64	
18	5/1/2021	Subways	103	28567	0.36	
19	5/1/2021	Buses	108	14730	0.73	
20	5/1/2021	Miscellaneous NYCT	4	4850	0.08	
21	5/1/2021	MTA Bus	23	3769	0.61	
22	6/1/2021	Subways	107	28541	0.37	
23	6/1/2021	Buses	103	14758	0.7	
24	6/1/2021	Miscellaneous NYCT	7	4808	0.15	
25	6/1/2021	MTA Bus	31	3772	0.82	
26	7/1/2021	Subways	112	28428	0.39	
27	7/1/2021	Buses	109	14858	0.73	
28	7/1/2021	Miscellaneous NYCT	4	4765	0.08	

<https://data.ny.gov/api/views/8vjt-4zv4/rows.csv?accessType=DO>

WNLOAD

1. Cleaning

	Month	Department	Accidents	Employees	Lost Time Rate
0	Jan 2023	Subways	122	29691	0.41
1	Feb 2023	Subways	105	29802	0.35
2	Mar 2023	Subways	120	29928	0.4
3	Apr 2023	Subways	99	30059	0.33
4	May 2023	Subways	114	30061	0.38
5	Jun 2023	Subways	103	30083	0.34
6	Jul 2023	Subways	97	30226	0.32
7	Aug 2023	Subways	104	30201	0.34
8	Sep 2023	Subways	95	30212	0.31
9	Oct 2023	Subways	89	30403	0.29
10	Nov 2023	Subways	111	30537	0.36
11	Dec 2023	Subways	60	30603	0.2

```
[ ] import csv
import pandas as pd
from google.colab import files

def data_parser(file_path):
    with open(file_path, 'r') as f:
        reader = csv.reader(f, delimiter=',')
        data = list(reader)
    df = pd.DataFrame(data[1:], columns=data[0])

    # df['Month'] = pd.to_datetime(df['Month'], format='%m/%d/%Y')
    # df = df[df['Month'].dt.year == 2023]
    # df['Month'] = df['Month'].dt.strftime('%b %Y')

    df = df.rename(columns={'Lost Time Accidents': 'Accidents',
                           'Lost Time Accident Rate': 'LostTimeRate'})

    df = df[df['Department'] == 'Subways']
    df.reset_index(drop=True, inplace=True)
    df.to_csv('cleaned_DownloadedData.csv', index=False)

    return df

##### Function Call #####
data_parser('Subways.csv')
```

2. Web Collection 1 (Wikipedia Page)

Community Board (CB)	Area km ²	Pop. Census 2010	Pop./km ²	Neighborhoods
Bronx CB 1	7.17	91,497	12,761	Melrose, Mott Haven, Port Morris
Bronx CB 2	5.54	52,246	9,792	Hunts Point, Longwood
Bronx CB 3	4.07	79,762	19,598	Claremont, Concourse Village, Crotona Park, Morrisania
Bronx CB 4	5.28	146,441	27,735	Concourse, Highbridge, Mount Eden
Bronx CB 5	3.55	128,200	36,145	Fordham, Morris Heights, Mount Hope, University Heights
Bronx CB 6	4.01	83,268	20,765	Bathgate, Belmont, East Tremont, West Farms
Bronx CB 7	4.84	139,286	28,778	Bedford Park, Fordham Heights, Jerome Park, Kingsbridge Heights, Norwood, University Heights
Bronx CB 8	8.83	101,731	11,521	Fieldston, Kingsbridge, Kingsbridge Heights, Marble Hill, Riverdale, Spuyten Duyvil, Van Cortlandt Village
Bronx CB 9	12.41	172,298	13,884	Bronx River, Bruckner, Castle Hill, Clason Point, Harding Park, Parkchester, Soundview, Unionport
Bronx CB 10	16.76	120,392	7,183	City Island, Co-op City, Locust Point, Pelham Bay, Silver Beach, Throgs Neck, Westchester Square
Bronx CB 11	9.32	113,232	12,149	Allerton, Bronxdale, Indian Village, Laconia, Morris Park, Pelham Gardens, Pelham Parkway, Van Nest
Bronx CB 12	14.56	152,344	10,463	Baychester, Edenwald, Eastchester, Fish Bay, Olinville, Wakefield, Williamsbridge, Woodlawn Heights
Brooklyn CB 1	12.82	160,338	12,507	Greenpoint, Williamsburg, Williamsburg Houses
Brooklyn CB 2	7.72	98,620	12,775	Boerum Hill, Brooklyn Heights, Brooklyn Navy Yard, Clinton Hill, Dumbo, Fort Greene, Fulton Ferry, Fulton Mall, Vinegar Hill

https://en.wikipedia.org/wiki/Neighborhoods_in_New_York_City

2. Cleaning

	CrashDate	CollisionID	Borough	PplInjured	PpleKilled
0	2022-01-01	4491355	Brooklyn	0	0
1	2022-01-01	4491216	Queens	2	0
2	2022-01-01	4491404	Bronx	0	0
3	2022-01-01	4491206	Manhattan	0	0
4	2022-01-01	4491170	Manhattan	0	0
...	...	...	...	...	...
14009	2022-10-04	4570509	Brooklyn	2	0
14010	2022-10-05	4570572	Manhattan	0	0
14011	2022-10-07	4571517	Brooklyn	0	0
14012	2022-10-07	4571528	Brooklyn	0	0
14013	2022-10-13	4573583	Queens	2	0
14014	2022-10-13	4573583	Queens	2	0

```
[22] import requests
      from bs4 import BeautifulSoup
      import pandas as pd
      import io

      def web_parser1(website):
          response = requests.get(website)
          html_content = response.text
          soup = BeautifulSoup(html_content, 'html.parser')
          table = soup.find('table', class_='wikitable')
          df = pd.read_html(io.StringIO(str(table)))[0]

          df.drop('Neighborhoods', axis=1, inplace=True)
          df.rename(columns={'Community Board (CB)': 'Borough',
                             'Pop. Census 2010': 'Population'}, inplace=True)

          # Clean up 'Borough' column by removing leading/trailing whitespace and non-alphanumeric characters
          df['Borough'] = df['Borough'].str.split('CB').str[0].str.replace(r'\W', '', regex=True).str.strip()

          df = df.iloc[1:-1]

          # Group by 'Borough', summing the population and area, and averaging the population density
          grouped_df = df.groupby('Borough').agg({'Area km2': 'sum', 'Population': 'sum', 'Pop./ km2': 'mean'}).reset_index()

          # Staten Island is not connected to NYC subway
          grouped_df = grouped_df.iloc[1:-1]

          # Round 'Pop./ km2' to 2 decimal places and format all numerical columns as integers
          grouped_df['Pop./ km2'] = grouped_df['Pop./ km2'].round(2)
          grouped_df[['Area km2', 'Population']] = grouped_df[['Area km2', 'Population']].astype(int)

          grouped_df.to_csv('cleaned_HTMLData.csv', index=False)

          return grouped_df

      ##### Function Call #####
      web_parser1('https://en.wikipedia.org/wiki/Neighborhoods_in_New_York_City')
```

3. Web Collection 2 (Car Crash Dataset)

Motor Vehicle Collisions - Crashes

CRASH DATE	CRASH TIME	BOROUGH	ZIP CODE	LATITUDE	LONGITUDE	LOCATION	ON STREET NAME
09/11/2021	2:39						WHITESTONE EXPRESSWAY
03/26/2022	11:45						QUEENSBORO BRIDGE UPPER
06/29/2022	6:55						THROGS NECK BRIDGE
09/11/2021	9:35	BROOKLYN	11208	40.667202	-73.8665	(40.667202, -73.8665	
12/14/2021	8:13	BROOKLYN	11233	40.683304	-73.917274	(40.683304, -73.9172	SARATOGA AVENUE
04/14/2021	12:47						MAJOR DEEGAN EXPRESSWAY RAMP
12/14/2021	17:05			40.709183	-73.956825	(40.709183, -73.9568	BROOKLYN QUEENS EXPRESSWAY
12/14/2021	8:17	BRONX	10475	40.86816	-73.83148	(40.86816, -73.83148	

1 to 50 of 2,083,227

https://data.cityofnewyork.us/Public-Safety/Motor-Vehicle-Collisions-Crashes/h9gi-nx95/data_preview

3. Cleaning

	CrashDate	CollisionID	Borough	PplInjured	PpleKilled
0	2022-01-01	4491355	Brooklyn	0	0
1	2022-01-01	4491216	Queens	2	0
2	2022-01-01	4491404	Bronx	0	0
3	2022-01-01	4491206	Manhattan	0	0
4	2022-01-01	4491170	Manhattan	0	0
...	...	...	...	...	...
14009	2022-10-04	4570509	Brooklyn	2	0
14010	2022-10-05	4570572	Manhattan	0	0
14011	2022-10-07	4571517	Brooklyn	0	0
14012	2022-10-07	4571528	Brooklyn	0	0
14013	2022-10-13	4573583	Queens	2	0

14014 ... up to 5 columns

```

import requests
import pandas as pd

def web_parser2(website):
    response = requests.get(website)
    data = response.json()
    df = pd.DataFrame(data)
    df.drop(['on_street_name', 'off_street_name', 'contributing_factor_vehicle_1',
            'contributing_factor_vehicle_2', 'vehicle_type_code1', 'vehicle_type_code2',
            'zip_code', 'vehicle_type_code3', 'vehicle_type_code4', 'latitude', 'longitude',
            'longitude', 'location', 'contributing_factor_vehicle_3', 'vehicle_type_code3',
            'contributing_factor_vehicle_4', 'contributing_factor_vehicle_5', 'vehicle_type_code_5',
            'vehicle_type_code_4', 'cross_street_name', 'crash_time'], axis=1, inplace=True)

    df = df.rename(columns={'crash_date': 'CrashDate',
                           'number_of_persons_injured': 'PersonsInjured',
                           'number_of_persons_killed': 'PersonsKilled',
                           'number_of_pedestrians_injured': 'PedestriansInjured',
                           'number_of_pedestrians_killed': 'PedestriansKilled',
                           'number_of_cyclist_injured': 'CyclistsInjured',
                           'number_of_cyclist_killed': 'CyclistsKilled',
                           'number_of_motorist_injured': 'MotoristsInjured',
                           'number_of_motorist_killed': 'MotoristsKilled',
                           'collision_id': 'CollisionID',
                           'borough': 'Borough'})

    df = df.dropna(subset=['Borough'])
    df.drop_duplicates(subset=['CollisionID'])

    df['CrashDate'] = pd.to_datetime(df['CrashDate'])
    df = df[df['CrashDate'].dt.year == 2022]

    df['Borough'] = df['Borough'].apply(lambda x: x.title())

    df['PersonsInjured'] = df['PersonsInjured'].astype(int)
    df['PersonsKilled'] = df['PersonsKilled'].astype(int)
    df['PedestriansInjured'] = df['PedestriansInjured'].astype(int)
    df['PedestriansKilled'] = df['PedestriansKilled'].astype(int)
    df['CyclistsInjured'] = df['CyclistsInjured'].astype(int)
    df['CyclistsKilled'] = df['CyclistsKilled'].astype(int)
    df['MotoristsInjured'] = df['MotoristsInjured'].astype(int)
    df['MotoristsKilled'] = df['MotoristsKilled'].astype(int)
    df['CollisionID'] = df['CollisionID'].astype(int)

    df['PplInjured'] = df[['PersonsInjured', 'PedestriansInjured', 'CyclistsInjured', 'MotoristsInjured']].astype(int).sum(axis=1)
    df['PpleKilled'] = df[['PersonsKilled', 'PedestriansKilled', 'CyclistsKilled', 'MotoristsKilled']].astype(int).sum(axis=1)

    df.drop(['PersonsInjured', 'PersonsKilled', 'PedestriansInjured', 'PedestriansKilled', 'CyclistsInjured',
            'CyclistsKilled', 'MotoristsInjured', 'MotoristsKilled'], axis=1, inplace=True)

    df.sort_values(by='CrashDate', inplace=True)

    df.reset_index(drop=True, inplace=True)
    df.to_csv('cleaned_APIData.csv', index=False)

    return df

##### Function Call #####
limit = 100000
nullLimit = 31415927
choice = limit
web_parser2("https://data.cityofnewyork.us/resource/m6gi-mx95.json?limit={choice}")

```

4. Additional Dataset (Subway Rider Dataset)

DATA.NY.GOV

OPEN NY CATALOG DEVELOPERS HELP ▾ ABOUT ▾

English ▾ Sign In

About Data Related Content

Actions ▾ Export

MTA Daily Ridership Data: Beginning 2020

Search X

Date ↑	Subways:...	Subways:...	Buses: T...	Buses: % ...	LIRR: Tot...	LIRR: % o...	Metro-No...	Metro-No...	Access:...	Access:...	Bridges a...	Bridges a...	Staten Isl...	Staten Is...
03/01/2020	2,212,965	97%	984,908	99%			55,826	59%	19,922	113%	786,961	98%	1,636	52%
03/02/2020	5,329,915	96%	2,209,066	99%	321,569	103%	180,702	66%	30,338	102%	874,620	95%	17,140	107%
03/03/2020	5,481,103	98%	2,228,608	99%	319,727	102%	190,648	69%	32,767	110%	882,175	96%	17,453	109%
03/04/2020	5,498,809	99%	2,177,165	97%	311,662	99%	192,689	70%	34,297	115%	905,558	98%	17,136	107%
03/05/2020	5,496,453	99%	2,244,515	100%	307,597	98%	194,387	70%	33,209	112%	929,298	101%	17,203	108%
03/06/2020	5,189,447	93%	2,066,743	92%	289,171	92%	205,056	74%	30,970	104%	945,408	103%	15,285	96%
03/07/2020	2,814,637	92%	1,249,085	94%	106,058	98%	75,839	56%	18,117	107%	827,908	95%	2,445	48%
03/08/2020	2,120,656	93%	957,163	96%	81,565	94%	60,800	64%	19,477	111%	765,084	95%	1,672	53%

1 to 50 of 1,517

https://data.ny.gov/Transportation/MTA-Daily-Ridership-Data-Beginning-2020/vxuj-8kew/data_preview

4. Cleaning

	Date	SubwayRiders	BusRiders	MetroRiders	RailRoadRiders
0	2023-01-01	1675507	475226	66309	67722
1	2023-01-02	1938154	739507	72404	76247
2	2023-01-03	3175777	1268913	167136	179109
3	2023-01-04	3413052	1391449	170847	182834
4	2023-01-05	3428520	1378698	166642	175278
...	...	...	...	...	...
360	2023-12-27	2912007	955890	186541	220010
361	2023-12-28	3064841	969231	193235	218546
362	2023-12-29	3198885	1007697	210361	237781
363	2023-12-30	2440211	720564	121647	121688
364	2023-12-31	1934651	548344	89554	99817

```
import requests
import pandas as pd

def extra_source(jsonEnd):
    response = requests.get(jsonEnd)
    data = response.json()
    df = pd.DataFrame(data)

    # Drop unnecessary columns
    df.drop(['subways_of_comparable_pre_pandemic_day', 'buses_of_comparable_pre_pandemic_day', 'metro_north_of_comparable_pre_pandemic_day',
            'access_a_ride_total_scheduled_trips', 'access_a_ride_of_comparable_pre_pandemic_day', 'bridges_and_tunnels_total_traffic',
            'bridges_and_tunnels_of_comparable_pre_pandemic_day', 'staten_island_railway_total_estimated_ridership',
            'staten_island_railway_of_comparable_pre_pandemic_day', 'lirr_of_comparable_pre_pandemic_day'], axis=1, inplace=True)

    # Rename columns to proper headers
    df.rename(columns={'date': 'Date',
                      'subways_total_estimated_ridership': 'SubwayRiders',
                      'buses_total_estimated_ridership': 'BusRiders',
                      'metro_north_total_estimated_ridership': 'MetroRiders',
                      'lirr_total_estimated_ridership': 'RailRoadRiders'}, inplace=True)

    # Drop NaN values from all columns
    df.dropna(inplace=True)

    # Convert Date column into dateTime format and filter for 2023 and format the date
    df['Date'] = pd.to_datetime(df['Date'])
    df = df[df['Date'].dt.year == 2023]

    # Cast all values into ints
    df['SubwayRiders'] = df['SubwayRiders'].astype(int)
    df['BusRiders'] = df['BusRiders'].astype(int)
    df['MetroRiders'] = df['MetroRiders'].astype(int)
    df['RailRoadRiders'] = df['RailRoadRiders'].astype(int)

    df.reset_index(drop=True, inplace=True)
    df.to_csv('cleaned_additionalData.csv', index=False)

    return df

##### Function Call #####
extra_source(f'https://data.ny.gov/resource/vxuj-6kew.json?$limit=(1517)')
```

Insight 1

```
import requests
import pandas as pd

def insight1():
    limit = 100000
    fulllimit = 53410227
    choice = fulllimit
    crashes = web_parser2(f'https://data.cityofnewyork.us/resource/h9gi-nx95.json?$limit={choice}')

    borough_crash_counts = crashes.groupby('Borough').size().reset_index(name='CrashCount')
    borough_crash_counts.sort_values(by='CrashCount', ascending=False, inplace=True)
    borough_crash_counts.reset_index(drop=True, inplace=True)
    borough_crash_counts = borough_crash_counts.iloc[:1]

    populationCount = web_parser1('https://en.wikipedia.org/wiki/Neighborhoods_in_New_York_City')

    merged_df = pd.merge(borough_crash_counts, populationCount, on='Borough', how='inner')

    merged_df['PeoplePerCrash'] = round(merged_df['Population'] / merged_df['CrashCount'], 2)
    merged_df.sort_values(by='PeoplePerCrash', ascending=True, inplace=True)
    merged_df.reset_index(drop=True, inplace=True)

    # Calculate the mean of numeric columns
    column_averages = merged_df.drop(columns=['Borough']).mean().to_frame().T
    column_averages['Borough'] = 'Average'

    # Concatenate the row containing column averages
    merged_df = pd.concat([merged_df, column_averages], ignore_index=True)

    return merged_df
```

	Borough	CrashCount	Area km2	Population	Pop./ km2	PeoplePerCrash
0	Brooklyn	22887.00	174.00	2465299.00	15847.390	107.720
1	Queens	17783.00	254.00	2240510.00	11220.570	125.990
2	Manhattan	11817.00	57.00	1529357.00	26506.920	129.420
3	Bronx	10500.00	96.00	1380697.00	17564.500	131.490
4	Average	15746.75	145.25	1903965.75	17784.845	123.655

Insight 2

```
[ ] import pandas as pd
    from datetime import datetime

    def insight2():
        # Load the public transportation ridership data
        pt_ridership = extra_source(f'https://data.ny.gov/resource/vxuj-8kew.json?&limit=(1517)')

        # Convert the 'Date' column to datetime format
        pt_ridership['Date'] = pd.to_datetime(pt_ridership['Date'])

        # Extract the month and year from the 'Date' column and create a new column with formatted strings
        pt_ridership['Month'] = pt_ridership['Date'].dt.strftime('%b %Y')

        # Group the data by month and sum the ridership values
        monthly_ridership = pt_ridership.groupby('Month').agg({'SubwayRiders': 'sum',
                                                                'BusRiders': 'sum',
                                                                'MetroRiders': 'sum',
                                                                'RailRoadRiders': 'sum'})

        # Reset the index to make 'Month' the first column and use the default numeric index
        monthly_ridership.reset_index(inplace=True)

        # Convert the 'Month' column to datetime objects for sorting
        monthly_ridership['Month'] = pd.to_datetime(monthly_ridership['Month'], format='%b %Y')

        # Sort the DataFrame by the 'Month' column
        monthly_ridership.sort_values(by='Month', inplace=True)
        monthly_ridership['Month'] = monthly_ridership['Month'].dt.strftime('%b %Y')

        # Reset the index to ensure it follows the sorted order
        monthly_ridership.reset_index(drop=True, inplace=True)

        lostSubwayTime = subway_lostTimeRate_df
        lostSubwayTime['Month'] = pd.to_datetime(lostSubwayTime['Month'], format='%m/%d/%Y')
        lostSubwayTime = lostSubwayTime[lostSubwayTime['Month'].dt.year == 2023]
        lostSubwayTime['Month'] = lostSubwayTime['Month'].dt.strftime('%b %Y')

        merged_df = pd.merge(monthly_ridership, lostSubwayTime, on='Month', how='inner')

        merged_df.drop(['BusRiders', 'MetroRiders', 'RailRoadRiders', 'Department', 'Employees', 'LostTimeRate'], axis = 1, inplace = True)

        merged_df['SubwayRiders'] = merged_df['SubwayRiders'].astype(int)
        merged_df['Accidents'] = merged_df['Accidents'].astype(int)

        merged_df['RidersPerAccident'] = round(merged_df['SubwayRiders'] / merged_df['Accidents'], 2)

        merged_df.sort_values(by='RidersPerAccident', ascending = False, inplace = True)
        merged_df.reset_index(drop=True, inplace=True)
```

	Month	SubwayRiders	Accidents	RidersPerAccident
0	Dec 2023	96887660	60	1614794.33
1	Oct 2023	104296170	89	1171867.08
2	Sep 2023	95330225	95	1003476.05
3	Apr 2023	94198922	99	951504.26
4	Jun 2023	97457013	103	946184.59
5	Jul 2023	90797995	97	936061.80
6	May 2023	104435152	114	916097.82
7	Aug 2023	93997299	104	903820.18
8	Nov 2023	98487231	111	887272.35
9	Mar 2023	101030692	120	841922.43
10	Feb 2023	84299462	105	802852.02
11	Jan 2023	88999287	122	729502.35

Insight 3

```
[ ] import pandas as pd
def insight3():
    limit = 100000
    fulllimit = 53410227
    choice = fulllimit
    crashes = web_parser2(f'https://data.cityofnewyork.us/resource/h9gi-nx95.json?$limit={choice}')

    # Reformating of the month column to allow for sorting and grouping
    crashes['CrashDate'] = pd.to_datetime(crashes['CrashDate'])
    crashes['Month'] = crashes['CrashDate'].dt.strftime('%b')
    crashes_by_month = crashes.groupby('Month').size().reset_index(name='CrashCount')
    crashes_by_month['Month'] = pd.to_datetime(crashes_by_month['Month'], format='%b')
    crashes_by_month.sort_values(by='Month', inplace=True)
    crashes_by_month['Month'] = crashes_by_month['Month'].dt.strftime('%b')

    # Calling public transportation ridership data from Additional Dataset
    pt_ridership = extra_source(f'https://data.ny.gov/resource/vxuj-8kew.json?$limit={1517}')
    pt_ridership.drop(['BusRiders'], axis=1, inplace=True)

    # Date formatting
    pt_ridership['Date'] = pd.to_datetime(pt_ridership['Date'])

    # Extract the month and year from the 'Date' column and create a new column with formatted strings
    pt_ridership['Month'] = pt_ridership['Date'].dt.strftime('%b')
    monthly_ridership = pt_ridership.groupby('Month').agg({'SubwayRiders': 'sum',
                                                            'MetroRiders': 'sum',
                                                            'RailRoadRiders': 'sum'})

    # Reset the index to make 'Month' the first column and use the default numeric index
    monthly_ridership.reset_index(inplace=True)

    merged_df = pd.merge(crashes_by_month, monthly_ridership, on='Month', how='inner')

    # Add a 'TotalPublicTransport' column summing subway, metro, and railroad ridership
    merged_df['TotalPublicTransport'] = merged_df[['SubwayRiders', 'MetroRiders', 'RailRoadRiders']].sum(axis=1)

    return merged_df

### Function Call ###
insight3()
```

	Month	CrashCount	SubwayRiders	MetroRiders	RailRoadRiders	TotalPublicTransport
0	Jan	4959	88999287	4156954	4529728	97685969
1	Feb	4520	84299462	3810260	4256489	92366211
2	Mar	5513	101030692	4674166	5201989	110906847
3	Apr	5195	94198922	4471810	4917537	103588269
4	May	6095	104435152	5182089	5616409	115233650
5	Jun	5611	97457013	5142382	5806179	108405574
6	Jul	5668	90797995	4721239	5418885	100938119
7	Aug	5511	93997299	4992583	5841314	104831196
8	Sep	5710	95330225	4914094	5606594	105850913
9	Oct	5996	104296170	5446892	6014657	115757719
10	Nov	5416	98487231	5339381	5889396	109716008
11	Dec	5517	96887660	5189979	5783396	107861035

Insight 4

```
##@title Insight 4
def insight4():
    limit = 100000
    fulllimit = 53410227
    choice = fulllimit
    crashes = web_parser2(f'https://data.cityofnewyork.us/resource/h9gi-nx95.json?$limit={choice}')

    # Group by Borough and sum the number of people injured and killed
    borough_stats = crashes.groupby('Borough').agg({'PplInjured': 'sum'}).reset_index()

    # Sort in descending order of number of people injured.
    borough_stats.sort_values(by='PplInjured', ascending = False, inplace = True)

    # Removal of Staten Island because they are not connected to the NYC Subway or Public Transportation
    borough_stats = borough_stats.iloc[:-1]
    borough_stats.reset_index(drop=True, inplace=True)

    return borough_stats

##### Function Call #####
insight4()
```

	Borough	PplInjured
0	Brooklyn	22338
1	Queens	16558
2	Bronx	11362
3	Manhattan	9965

Insight 5

```
import pandas as pd

def insight5():
    # Assuming you already have a DataFrame named lostSubwayTime
    lostSubwayTime = data_parser('Subways.csv')

    # Convert 'Month' column to datetime type
    lostSubwayTime['Month'] = pd.to_datetime(lostSubwayTime['Month'])

    # Filter the DataFrame for the year 2023
    lostSubwayTime = lostSubwayTime[lostSubwayTime['Month'].dt.year == 2023]

    # Convert 'Employees' column to integer type
    lostSubwayTime['Employees'] = lostSubwayTime['Employees'].astype(int)

    # Calculate the average number of employees affected
    avg_employees_affected = round(lostSubwayTime['Employees'].mean(), 2)

    # Call of total Crash Data
    limit = 100000
    fullLimit = 53410227
    choice = limit
    crashes = web_parser2(f'https://data.cityofnewyork.us/resource/h9gi-nx95.json?\$limit={choice}')

    # Calculating Average of people injured per accident
    avg_people_injured = round(crashes['PplInjured'].mean(), 2)

    # Create DataFrame
    average_df = pd.DataFrame({
        'Average Employee Affected': [avg_employees_affected],
        'Average People Injured': [avg_people_injured],
    })

    return average_df

# Function call
insight5()
```

	Average Employee Affected	Average People Injured
--	---------------------------	------------------------

0	30150.5	1.71
---	---------	------

Visualization 1

```
##title Visualization 1
import pandas as pd
import matplotlib.pyplot as plt
```

```
def visual1():
    crash_rider_df = insight3()
    plt.figure(figsize=(10, 6))

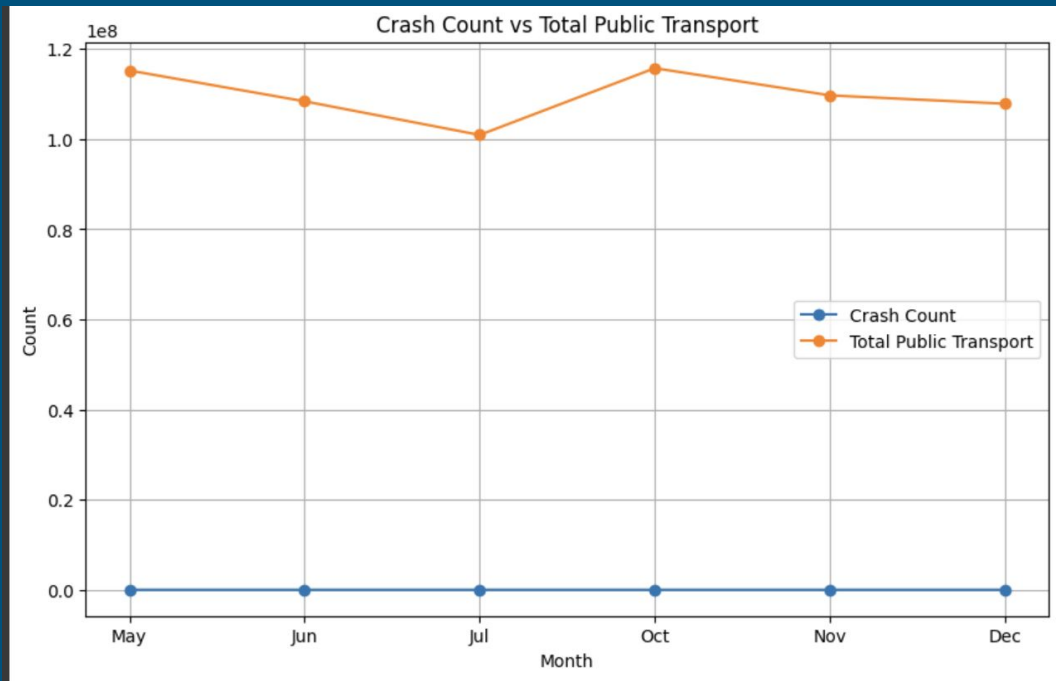
    # Plotting CrashCount
    plt.plot(crash_rider_df['Month'], crash_rider_df['CrashCount'], markers='o', label='Crash Count')

    # Plotting TotalPublicTransport
    plt.plot(crash_rider_df['Month'], crash_rider_df['TotalPublicTransport'], markers='o', label='Total Public Transport')

    # Adding labels and title
    plt.xlabel('Month')
    plt.ylabel('Count')
    plt.title('Crash Count vs Total Public Transport')
    plt.legend()

    # Displaying the plot
    plt.grid(True)
    plt.show()

##### Function Call #####
visual1()
```



Visualization 2

```
##@title Visualization 2
import matplotlib.pyplot as plt

def visual2():
    riderDF = extra_source(f'https://data.ny.gov/resource/vxuj-8kew.json?$limit={1517}')

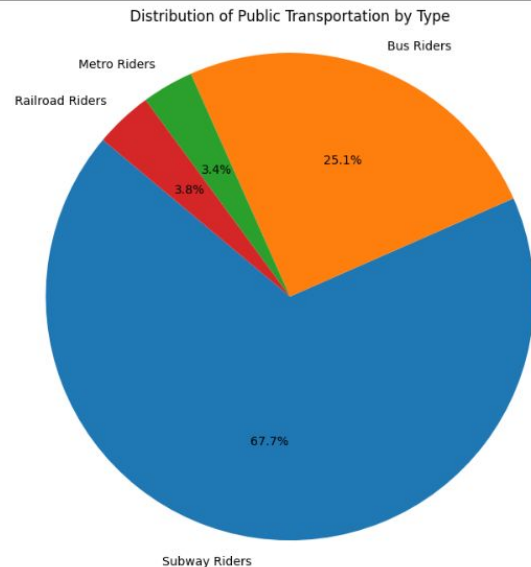
    # Calculate the total ridership for each type
    total_subway_riders = riderDF['SubwayRiders'].sum()
    total_bus_riders = riderDF['BusRiders'].sum()
    total_metro_riders = riderDF['MetroRiders'].sum()
    total_railroad_riders = riderDF['RailRoadRiders'].sum()

    # Create labels and sizes for the pie chart
    labels = ['Subway Riders', 'Bus Riders', 'Metro Riders', 'Railroad Riders']
    sizes = [total_subway_riders, total_bus_riders, total_metro_riders, total_railroad_riders]

    # Plotting the pie chart
    plt.figure(figsize=(10, 8))
    plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=140)
    plt.title('Distribution of Public Transportation by Type')
    plt.axis('equal') # Equal aspect ratio ensures that pie is drawn as a circle

    # Show the pie chart
    plt.show()

# Function call
visual2()
```



Visualization 3

```
import matplotlib.pyplot as plt
def visual3():
    population_df = web_parser1('https://en.wikipedia.org/wiki/Neighborhoods_in_New_York_City')

    # Plotting
    plt.figure(figsize=(10, 6))

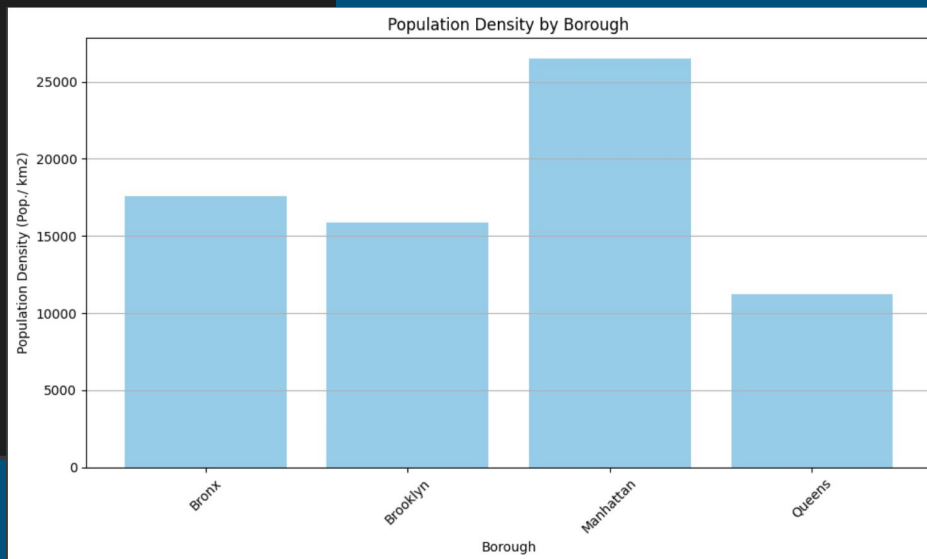
    # Creating bars
    plt.bar(population_df['Borough'], population_df['Pop./ km2'], color='skyblue')

    # Adding labels and title
    plt.xlabel('Borough')
    plt.ylabel('Population Density (Pop./ km2)')
    plt.title('Population Density by Borough')

    # Rotating x-axis labels for better readability
    plt.xticks(rotation=45)

    # Displaying the plot
    plt.grid(axis='y')
    plt.tight_layout()
    plt.show()

##### Function Call #####
visual3()
```



Conclusion

After analyzing 4 different datasets and viewing the numbers associated with car crashes and the number of public transportation riders in the months in 2023, we can see that there isn't a correlation between traffic accidents leading people to use public transportation.